

Введение

Актуальность:

Наиболее значимое влияние на здоровое тело оказывает правильное питание, которым нередко многие пренебрегают. Именно питание играет ключевую роль в профилактике заболеваний и обеспечении нормального функционирования организма, снижает риск развития многих хронических заболеваний, влияет на работу сердечно-сосудистой системы, эндокринную систему, желудочно-кишечный тракт, а также помогает контролировать вес и поддерживать общее здоровье.

Среди населения наблюдается тренд на здоровый образ жизни. По статистике, 38% опрошенных за последний год перешли на более полезное питание. Опрос населения за 2025 год показывает, что 10% россиян ведут подсчет КБЖУ, а 25% учитывают примерное количество, что подтверждает заинтересованность пользователей в удобных инструментах для контроля рациона.

Цель работы:

Разработка Telegram-бота для автоматизированного учёта питания, обеспечивающего расчет КБЖУ на основе введенного пользователем продукта и веса порции с сохранением истории в базе данных.

Задачи:

- Проанализировать предметную область и аналоги.
- Спроектировать архитектуру системы.
- Разработать алгоритм нечеткого поиска продуктов.
- Реализовать хранение данных в PostgreSQL.
- Разработать пользовательский интерфейс на основе inline-кнопок.
- Обеспечить развертывание системы через Docker.
- Провести тестирование.

Объект и предмет исследования:

- Объект: процесс учета и анализа питания пользователя.
- Предмет: программные средства и алгоритмы реализации Telegram-бота для ведения дневника питания.

Практическая значимость:

- Бот работает и решает поставленную задачу — предоставить новым пользователям возможность попробовать начать учитывать состав своего питания.
- Не требует дополнительных приложений, работает внутри Telegram (модульная архитектура допускает интеграцию с другими платформами).
- Реализован алгоритм неточного ввода, упрощающий внесение приемов пищи.
- Большой потенциал для масштабирования и добавления нового функционала.

Структура работы:

Выпускная квалификационная работа состоит из введения, четырех глав, заключения, списка литературы и приложений.

В первой главе проводится анализ предметной области: рассматривается понятие сбалансированного питания и подсчета КБЖУ, анализируются существующие программные решения, выявляются их недостатки и формулируются требования к разрабатываемой системе.

Во второй главе описывается проектирование системы: обосновывается выбор технологического стека, разрабатывается архитектура приложения, проектируется схема базы данных и строятся UML-диаграммы, отражающие структуру и поведение системы.

В третьей главе излагается реализация основных компонентов: алгоритм нечеткого поиска продуктов на основе расстояния Левенштейна, расчет КБЖУ пропорционально весу порции, работа с базой данных PostgreSQL, пользовательский интерфейс на основе inline-кнопок, а также развертывание приложения с использованием Docker.

В четвертой главе представлены результаты тестирования: модульные тесты сервисного слоя и проверка ключевых пользовательских сценариев.

В заключении подводятся итоги работы и намечаются направления дальнейшего развития системы.

Глава 1. Анализ предметной области

1.1. Принципы здорового питания и понятие КБЖУ

Здоровое питание — это сбалансированный рацион, который обеспечивает организм всеми необходимыми питательными веществами. Оно включает в себя несколько принципов:

- Достаточность — рацион удовлетворяет, но не превышает потребности организма в макро- и микронутриентах, предотвращая дефицит любого из них.
- Сбалансированность — общее потребление калорий соответствует энергозатратам организма при поддержании надлежащего соотношения между тремя основными источниками энергии – белками, жирами и углеводами.
- Умеренность — ограниченное потребление питательных веществ, ингредиентов и продуктов питания, которые могут наносить вред здоровью.
- Разнообразие — включение в рацион широкого спектра продуктов в пределах каждой группы, а также за счет сочетания различных групп.

Эти принципы подчеркивают важную мысль: существует понятие калорийности и нутриентного состава пищи, и необходимо соблюдать баланс между этими составляющими.

Для удобства контроля питания используется дневник питания — систематическая запись всего, что человек ест и пьет в течение дня. Такой метод повышает осознанность в питании и выявляет привычки, о которых мы можем не подозревать. Однако эффективность дневника сводится к нулю, если не заниматься его наполнением.

1.2. Обзор существующих решений и их недостатки

MyFitnessPal — одно из самых популярных приложений на разных платформах, имеет большое количество положительных отзывов от пользователей и богатый функционал. Однако можно выделить несколько недостатков:

- Это отдельное приложение, требующее скачивания и свободного места.
- Бесплатная версия перегружена предложениями по покупке улучшенной версии.
- Принудительная регистрация аккаунта пользователя с прохождением анкетирования для выявления потребностей и мотивации, сбор данных о весе, росте, стране проживания, активности.
- Перегруженный интерфейс.
- Неудобный ввод порции.

FatSecret — еще одно популярное приложение, главным преимуществом которого является активное сообщество пользователей, публикующих свой прогресс в ленте новостей, что мотивирует других пользователей.

Недостатки FatSecret:

- Это отдельное приложение, требующее скачивания и свободного места.
- Бесплатная версия перегружена предложениями по покупке улучшенной версии.
- Принудительная регистрация аккаунта пользователя с прохождением анкетирования для выявления потребностей и мотивации, сбор данных о весе, росте, стране проживания, активности.

Все эти недостатки не перечеркивают полезности и проработанности приложений, однако затрудняют вход для новых пользователей, которые хотят попробовать учитывать КБЖУ и привить себе новую привычку.

1.3. Обоснование выбранного подхода

Крупные коммерческие приложения предлагают пользователю избыточный функционал: распознавание по фото, социальную ленту или сложную аналитику. Однако ключевым

барьером для регулярного ведения дневника питания остается сложность и временные затраты на процесс фиксации приёма пищи.

Разработанный в рамках данной работы бот реализует минималистичный, но достаточный подход: текстовый ввод с последующим извлечением ключевых параметров и сохранением результатов в базу данных. Такой подход позволяет пользователям эмоционально упростить контроль, так как он схож с обычной личной перепиской, что снижает психологический барьер — пользователь остается в привычной среде.

1.4 Технические требования

Функциональные требования

Учет питания:

- Система должна принимать текстовый ввод вида «[название продукта] [вес в граммах]»
- Система должна извлекать название продукта и вес порции из сообщения пользователя
- Система должна выполнять поиск продукта по базе данных
- При нескольких совпадениях система должна предложить пользователю выбор через inline-кнопки
- Система должна автоматически рассчитывать КБЖУ пропорционально указанному весу порции
- Система должна сохранять запись о приеме пищи в базе данных с привязкой к Telegram ID пользователя

Просмотр данных:

- Система должна формировать отчет за текущий день с перечнем всех записей и суммарными показателями КБЖУ
- Система должна формировать отчет за предыдущий день

Редактирование:

- Система должна предоставлять возможность удаления отдельных записей из дневника

Интерфейс:

- Система должна поддерживать команды /start и /menu
- Навигация должна осуществляться через inline-кнопки без необходимости ввода команд вручную

Нефункциональные требования

Надежность:

- Данные пользователей должны храниться в реляционной базе данных PostgreSQL
- Каждая запись должна содержать временную метку для корректной фильтрации по дате

Безопасность:

- Данные пользователей должны быть изолированы по Telegram ID
- Все запросы к базе данных должны использовать параметризованные выражения (защита от SQL-инъекций)

Развертывание:

- Приложение должно быть контейнеризировано с помощью Docker
- Конфигурация (токен бота, параметры БД) должна храниться в отдельном файле и не попадать в репозиторий

Глава 2.

Технологический стек

Golang — современный компилируемый строго типизированный язык программирования разработанный компанией Google в 2009 году, обладает рядом преимуществ

- **Встроенные инструменты параллелизма:** одной из главных особенностей являются легковесные потоки - горутины, которые способны высокоэффективно обрабатывать большое количество запросов и безопасно их передавать с помощью каналов.
- **Лаконичный и понятный синтаксис** — в языке нет наследования и классов
- **Обширная стандартная библиотека** покрывающая большое количество задач таких как: работа с сетевыми протоколами, подключения к базам данных, чтение JSON, логирование
- **Кросс-платформенность** — возможность создавать решения работающие на разных операционных системах

Telegram Bot API и go-telegram-bot-api

Telegram Bot API — это HTTP-based интерфейс, предоставляемый Telegram для взаимодействия сторонних приложений с платформой Telegram. API позволяет ботам получать обновления (входящие сообщения, нажатия на кнопки и т.д.), отправлять сообщения, управлять клавиатурами, обрабатывать файлы и выполнять другие действия . Взаимодействие осуществляется через отправку HTTP-запросов.

go-telegram-bot-api — официально поддерживаемая библиотека на языке go. Реализует полный набор API методов, что избавляет разработчиков от ручного написания HTTP-запросов к Telegram Bot API

PostgreSQL — это мощная реляционная база данных с открытым исходным кодом, которую можно использовать в различных сценариях, на различных операционных системах. Безопасность СУБД обеспечивается технологией транзакций - операции или действия, которые происходят полностью или не происходят вовсе. PostgreSQL подходит как для маленьких проектов, так и для масштабных, что делает ее достаточной для разработки telegram бота.

Docker — платформа позволяющая контейнировать приложение со всеми необходимыми для работы зависимостями и запускать его в изолированной среде, а также хранить образы приложений с помощью Docker Hub. В современных реалиях все чаще используется как стенд для разработки и деплой на масштабируемые кластеры происходит через другие инструменты. Тем не менее для небольших проектов Docker все еще достаточен для деплоя.

2.2 Архитектура системы

При проектировании архитектуры был выбран принцип разделения ответственности, который является ключевым в архитектурной философии Go — каждый слой решает свою задачу и не знает о деталях реализации соседнего. Такой подход упрощает внесение изменений: замена базы данных или способа отправки сообщений затрагивает только один слой, не ломая остальные.

Приложение состоит из четырёх слоёв:

- **Handler** — точка входа для всех входящих обновлений от Telegram. Принимает сообщения и callback-запросы, определяет их тип и передаёт управление нужному обработчику. Не содержит бизнес-логики.
- **Service** — слой бизнес-логики. Здесь происходит поиск продукта по базе, расчёт КБЖУ для указанной порции, формирование текста сообщений и построение inline-клавиатур.
- **Repository** — слой работы с данными. Отвечает за все обращения к PostgreSQL: сохранение записей о приёме пищи, выборку данных за день, удаление записей. Реализован через интерфейс, что позволяет при необходимости заменить хранилище без изменения остального кода.
- **Storage** — встроенная база продуктов питания, хранящаяся в памяти в виде статического массива. Содержит около 75 наименований с заранее указанными значениями КБЖУ на 100 граммов.

Взаимодействие слоёв однонаправленное: Handler → Service → Repository. Это исключает обратные зависимости и упрощает тестирование каждого компонента в отдельности.

Для управления состоянием сессии пользователя используется словарь **map[int64]UserState**, хранящийся в памяти. Ключом выступает Telegram ID пользователя, значением состояние пользователя. Пользователь может пребывать лишь в двух состояниях: просмотр меню и состояние по умолчанию ожидания ввода приема. Такое решение позволило минимизировать время ввода, так как бот всегда ждет сообщения от пользователя.

2.3 Проектирование базы данных

База данных включает одну таблицу — **food_log**, в которой хранится история всех приёмов пищи пользователей.

Поле	Тип	Описание
id	SERIAL PRIMARY KEY	Уникальный идентификатор записи
user_id	BIGINT	Telegram ID пользователя
food_name	VARCHAR(255)	Название продукта
calories	SMALLINT	Калорийность порции (ккал)
proteins	SMALLINT	Белки порции (г)

fats	SMALLINT	Жиры порции (г)
carbs	SMALLINT	Углеводы порции (г)
weight	SMALLINT	Вес порции (г)
created_at	TIMESTAMP TZ	Дата и время записи (UTC)

Поле **created_at** заполняется автоматически при вставке строки. Оно используется для фильтрации записей при формировании дневного отчёта: запросы ограничиваются временным диапазоном от начала до конца нужного дня в UTC.

Хранение данных в UTC является осознанным решением — это упрощает работу с часовыми поясами и исключает ошибки при конвертации дат на стороне приложения.

2.4 UML-диаграммы

Для формального описания системы были построены пять UML-диаграмм с помощью PlantUML.

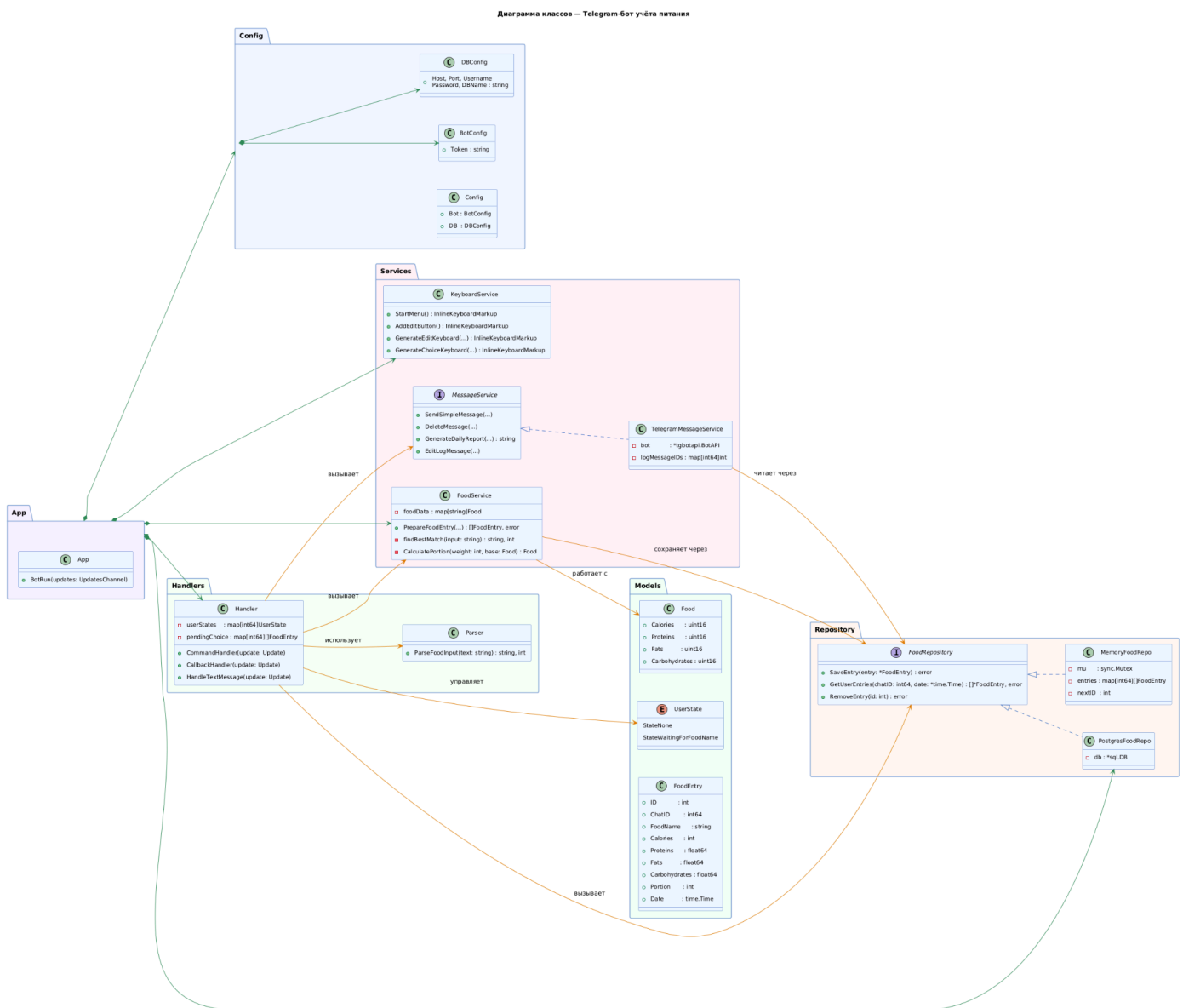
2.4.1 Диаграмма вариантов использования

Диаграмма вариантов использования показывает единственного актора системы - пользователя и семь доступных сценариев: запуск бота, открытие меню, добавление записи о еде, выбор продукта из нескольких вариантов, просмотр отчета за сегодня, просмотр отчета за вчера и удаление записи. Показательна связь **extend** между добавлением еды и выбором продукта: выбор из нескольких вариантов активируется только тогда, когда поиск возвращает неоднозначный результат, а не при каждом вводе.



2.4.2 Диаграмма классов

Диаграмма классов отражает шесть пакетов: **Config, Models, Repository, Services, Handlers, App**. Ключевой момент - **Repository** и **MessageService** оформлены как интерфейсы, а конкретные реализации (**PostgresFoodRepo**, **TelegramMessageService**) подключаются через них. Это значит, что при необходимости можно подменить хранилище или способ отправки сообщений, не трогая остальную логику. Для тестирования в диаграмме предусмотрена **MemoryFoodRepo** — реализация репозитория в памяти без обращения к реальной базе данных.

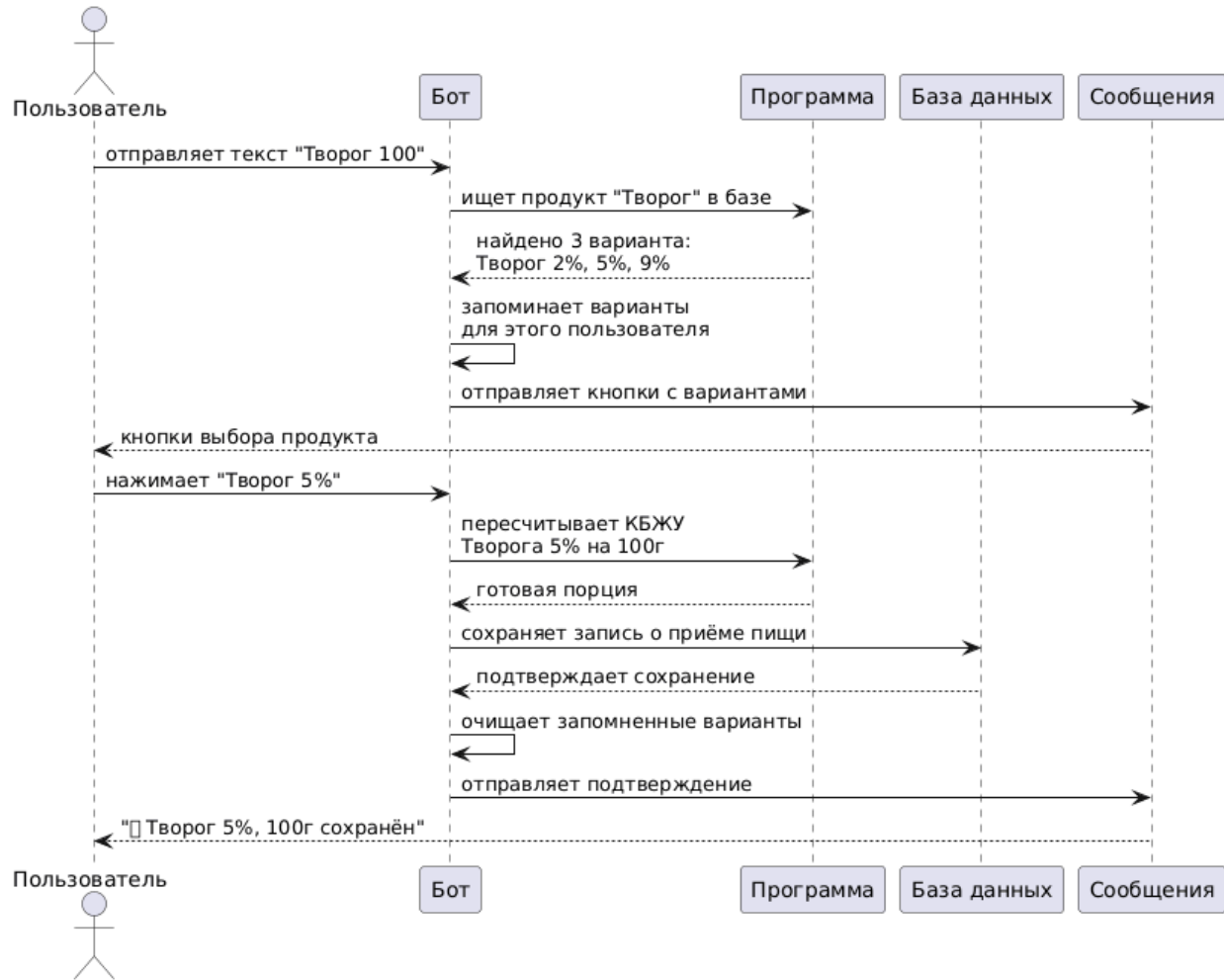


2.4.3 Диаграмма последовательности

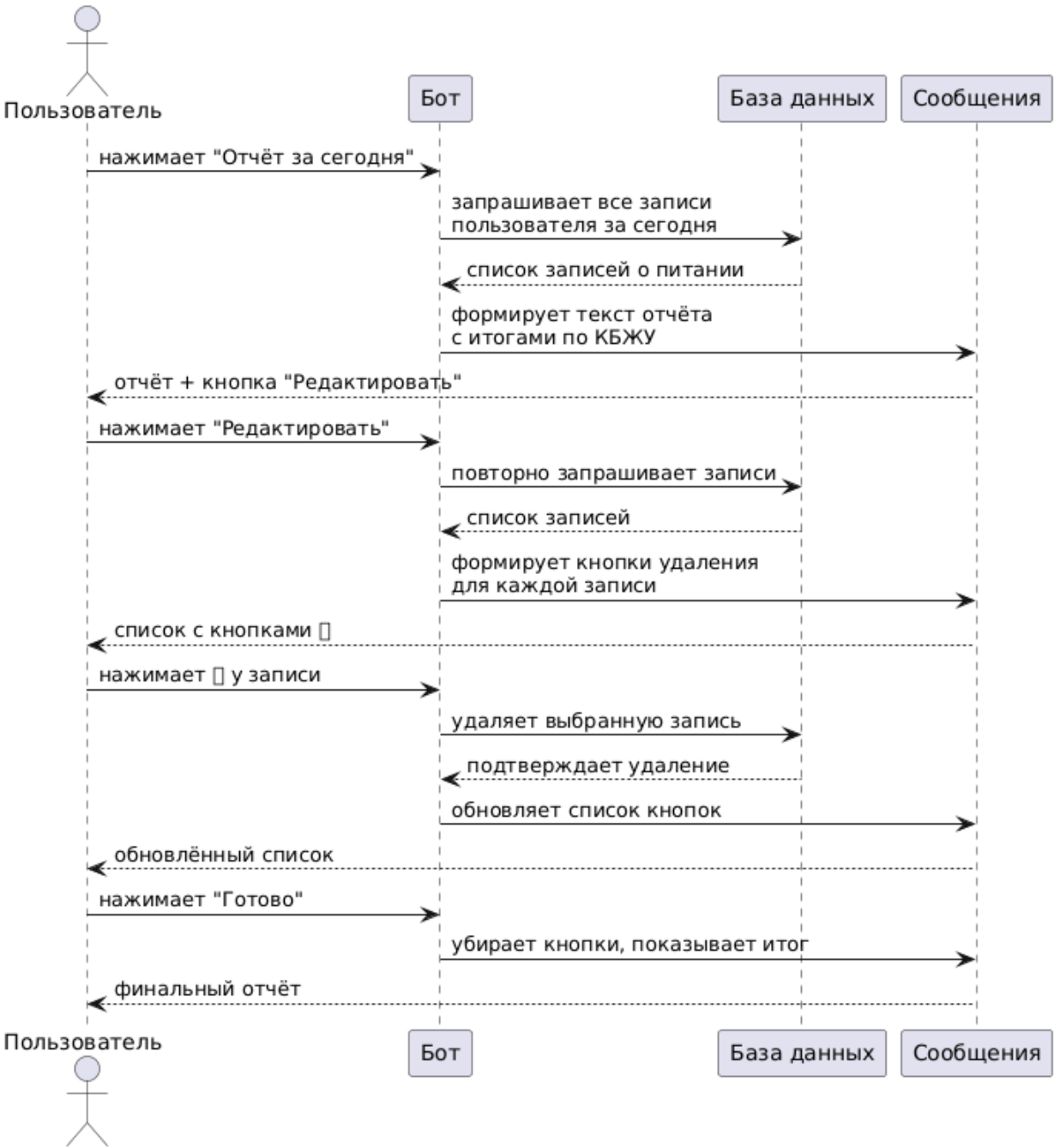
Диаграмма последовательности показывает полный жизненный цикл запроса от ввода пользователем текста до сохранения записи в базе данных. Включает три сценария поиска: точное совпадение, несколько совпадений с выбором через кнопки, продукт не найден.



Сценарий 2 — Несколько совпадений: выбор продукта

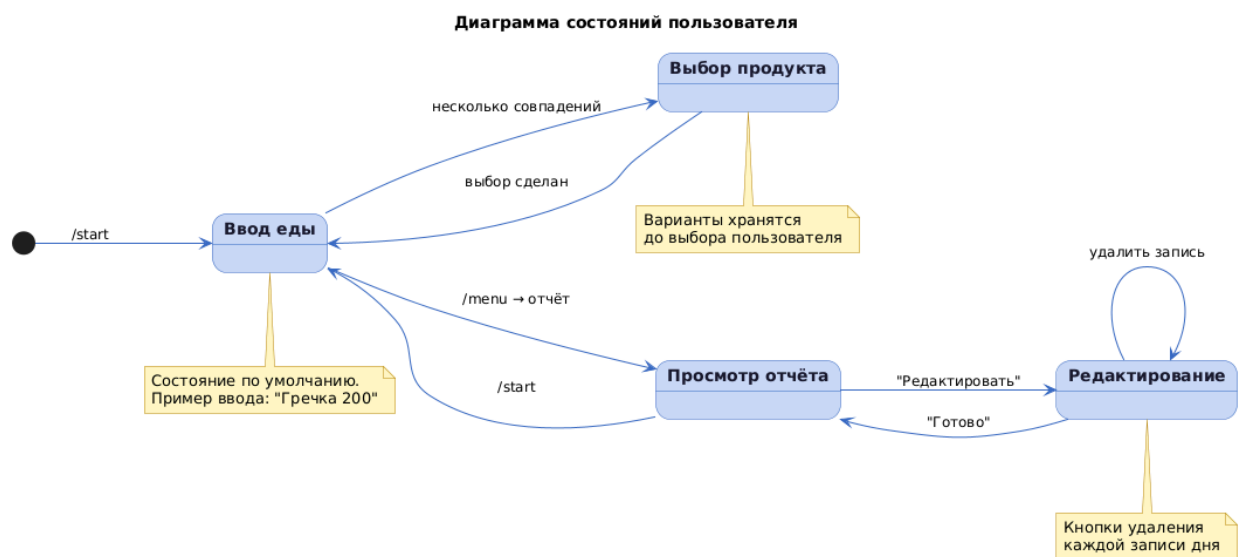


Сценарий 3 — Просмотр отчёта и редактирование



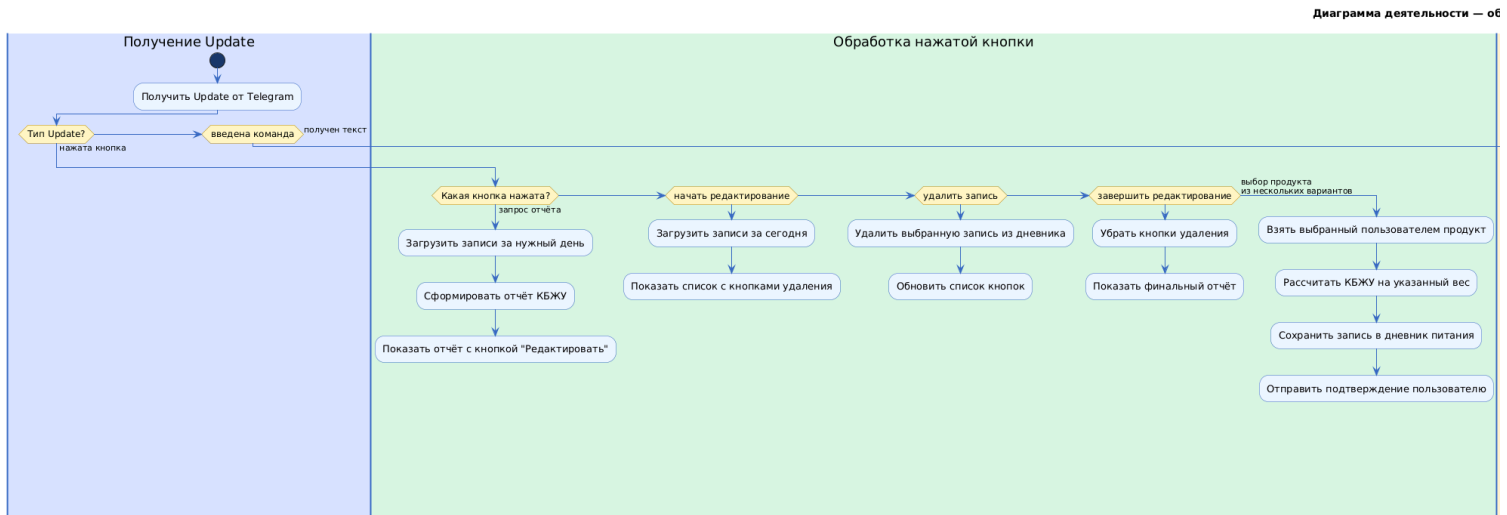
2.4.4 Диаграмма состояний

Диаграмма состояний описывает четыре состояния пользовательской сессии: «Ввод еды», «Выбор продукта», «Просмотр отчёта», «Редактирование». Начальное состояние — «Ввод еды», в котором бот находится большую часть времени. Переход в «Выбор продукта» происходит только при неоднозначном поиске, и после выбора сессия немедленно возвращается назад. Состояние «Редактирование» является замкнутым: каждое нажатие кнопки удаления оставляет пользователя в том же состоянии, пока он сам не нажмёт «Готово».

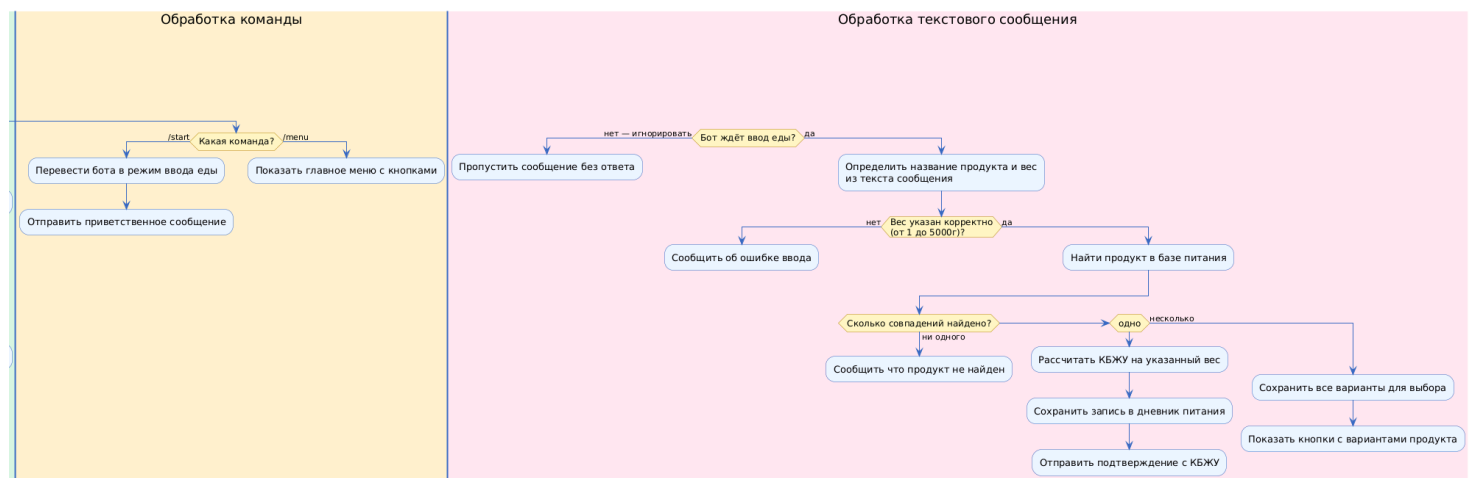


2.4.5 Диаграмма деятельности

Диаграмма деятельности построена в виде дорожек и разбирает обработку каждого входящего Update от Telegram. Первое ветвление определяет тип: нажатие кнопки, команда или текст. Для текстовых сообщений добавлена валидация веса, значения вне диапазона от 1 до 5000 граммов отклоняются с сообщением об ошибке ещё до обращения к базе продуктов. Дерево решений для поиска продукта явно показывает три исхода: ни одного совпадения, одно совпадение и несколько — каждый с отдельной ветвью обработки.



Обработка входящего сообщения



3 Глава

3.1 Структура проекта

```
tg-bot/
├── main.go          # Точка входа, сборка зависимостей, запуск бота
├── go.mod           # Описание модуля и зависимостей
├── Dockerfile       # Инструкция сборки образа
├── docker-compose.yml # Оркестрация бота и базы данных
├── config.yml       # Конфигурация (не в репозитории)
├── init.sql         # SQL-схема базы данных
├── storage/
│   └── data.go      # Встроенная база продуктов (~75 позиций)
├── internal/
│   ├── config/
│   │   └── config.go # Загрузка конфигурации через Viper
│   ├── models/
│   │   └── models.go # Структура FoodEntry
│   ├── domain/
│   │   └── state.go  # Перечисление состояний пользователя
│   ├── repository/
│   │   ├── food_repository.go # Интерфейс FoodRepository
│   │   └── postgres_food_repository.go # Реализация работы с PostgreSQL
│   ├── service/
│   │   ├── food.go   # Поиск продукта, расчёт КБЖУ
│   │   ├── keyboard.go # Построение inline-клавиатур
│   │   ├── messages.go # Отправка и редактирование сообщений
│   │   └── food_test.go # Юнит-тесты сервисного слоя
│   ├── utils/
│   │   └── parser.go  # Парсинг текстового ввода пользователя
│   └── bot/
│       └── handlers/
│           ├── handler.go # Инициализация обработчиков
│           ├── command_handlers.go # Обработка команд /start, /menu
│           └── callback_handlers.go # Обработка нажатий inline-кнопок
```

Описание структуры

main.go - здесь происходит сборка проекта, загрузка переменных из config.yml (содержит токен и данные для подключения к базе данных) для подключения к базе данных, соединение с базой данных.

storage/ - хранит файл data.go, в котором храниться **map[string]Food** словарь где ключ название продукта, значение КБЖУ на 100 г. Был выбран именно такой способ хранения продуктов, так как этот список не предполагает частого изменения, тем не менее допускается перенос этого словаря в реляционную базу данных.

internal/ - директория в которой содержится вся бизнес логика: распознавание продуктов из текстового ввода, поиск продуктов, парсинг сообщения, перерасчет порции продукта, отправка сообщений пользователю, генерация кнопок, сохранение в базу данных и многое другое.

Dockerfile и docker-compose.yaml - файлы для создания образа и сборки приложения в контейнер

init.sql - sql запрос, который создает таблицу в базе данных при первом запуске контейнера с приложением.

models/models.go - хранит в себе структуру одной записи продукта, благодаря ей происходит корректная отправка данных в базу данных и расчет на порцию.

repository/ - хранит интерфейс для хранилищ, который описывает какие методы и зависимости должны быть у модуля, который является хранилищем. Это важная особенность языка Go, так как компилятор сам распознает соответствует ли модуль интерфейсу, этот механизм реализует концепцию “Утиной типизации” - “Если что-то выглядит как утка, плавает как утка и крикает как утка, то, скорее всего, это утка.” .

3.2 Парсинг пользовательского ввода

После того как пользователь отправляет сообщение Telegram API передает Update, который хранит в себе много информации например chatID или текст сообщения. Если флаг состояния пользователя StateWaitingForFoodName то все сообщения пользователя направляются в handler/command_handler.go функция HandleTextMessage, которая вызывает парсер.

Парсер разбивает сообщение на отдельные слова и формирует слайс. Далее парсер находит в этом слайсе число и отделяет название от веса порции. Проверка на число производится с помощью дополнительной функции isNumber.

```
func ParseFoodInput(text string) (string, int) {
    words := strings.Fields(text)
    for i := len(words) - 1; i >= 0; i-- {
        if isNumber(words[i]) {
            digit, _ := strconv.Atoi(words[i])
            name := strings.Join(words[:i], " ")
            // Результат отправляется в виде двух переменных
            return name, digit
        }
    }
    // Или если число не найдено сообщение идет дальше вес порции будет 100г
    return strings.Join(words, " "), 100
}

func isNumber(word string) bool {
    _, err := strconv.Atoi(word)
    return err == nil
}
```

Парсер возвращает название и вес назад в HandleTextMessage, если вес будет больше 5000 грамм пользователю будет отправлено сообщение о недопустимом весе порции.

3.3 Алгоритм нечеткого поиска продуктов

После того как парсер извлек название продукта и его вес, FoodService, а именно функция PrepareFoodEntry ищет его в словаре. Поиск выполняется в три последовательных шага. Каждый следующий запускается только если предыдущий не дал результата.

Шаг 1 точное совпадение.

Название из ввода приводится к нижнему регистру и напрямую ищется в словаре. Если совпадение найдено — результат возвращается немедленно, дальнейшие шаги не выполняются.

Шаг 2 поиск по подстроке.

Если точного совпадения нет, алгоритм перебирает все продукты и проверяет, содержит ли название продукта введенную строку. Например, по запросу «творог» будут найдены «Творог 2%», «Творог 5%» и «Творог 9%». Все найденные варианты возвращаются сразу.

Шаг 3 расстояние Левенштейна.

Если поиск подстроки не дал результатов, значит либо в названии продукта, которое вернул парсер есть опечатка либо такого продукта нет в базе. Поэтому было принято решение реализовать неточный поиск с помощью расстояния Левенштейна.

Расстояние Левенштейна - это метрика сходства двух строковых последовательностей, чем меньше это значение тем больше сходства между двумя строками. Расстояние представляет из себя количество одно символьных операций (удаление, вставка или замена) для того чтобы преобразовать из одной строки другую.

Пример:

```
dist := Levenshtein.ComputeDistance("МОЛОКО", "МАЛАКО")  
// dist = 2, так как потребовалось две замены "А" на "О"
```

Метрика названа в честь советского математика, выпускника мехмата МГУ Владимира Иосифовича Левенштейна. Он всю жизнь проработал в Институте Прикладной Математики им. М.В.Келдыша, умер в 2017 году.

Мной самостоятельно не был реализован алгоритм вычисления расстояния Левенштейна, была использована сторонняя библиотека **levenshtein**.

Функция запускает поиск по базе и составляет слайс найденных совпадений при этом есть возможность менять предельное значение количества опечаток, изменив значение `maxDistance` с 1, например на 2.

```
const maxDistance = 1
results := make([]struct {
    name      string
    food      storage.Food
    distance  int
}, 0)

for name, food := range s.foodData {
    lname := strings.ToLower(name)
    dist := levenshtein.ComputeDistance(input, lname)
    if dist <= maxDistance {
        results = append(results, struct {
            name      string
            food      storage.Food
            distance  int
        }) {
            name:      name,
            food:      food,
            distance: dist,
        })
    }
}

// Сортируем нечеткие совпадения
sort.Slice(results, func(i, j int) bool {
    return results[i].distance < results[j].distance
})

if len(results) == 0 {
    return nil, fmt.Errorf("продукт не найден по запросу: %s", input)
}

return results, nil
}
```

3.4 Расчет КБЖУ

Словарь продуктов хранит значения КБЖУ на 100 г. Парсер отделил размер порции от названия именно для этого этапа. Если обратить внимание что из себя представляет results, то можно заметить, что это слайс, элементы которого структуры, хранящие в себе название продукта, элемент из словаря продукта и расстояние Левенштейна. Этот слайс возвращается в функцию PrepareFoodEntry, в которой вычисляется КБЖУ для всего слайса продуктов обычным последовательным перемножением всех элементов на вес и возвращается слайсом entry, готовым к сохранению в базу данных назад в handler.

Как только entry передается назад в handler в зависимости от количества результатов запускается один из трех сценариев:

1. Слайс результатов entry пуст - пользователю будет отправлено сообщение об ошибке при добавлении продукта
2. Один элемент слайса entry - один результат, который будет отправлен в базу данных, пользователь увидит сообщение, что все успешно сохранено. Пример такого сообщения: “Запись добавлена: **Лосось (300 г).**”
3. Слайс с результатами содержит несколько элементов - после этого будет запущен процесс генерации и отправки пользователю inline-кнопок, их количество и содержание текста на них прямо зависит от количества и содержания элементов слайса entry. Как только пользователь нажмет на одну из таких кнопок, ему придет такое же сообщение как из второго сценария.

3.5 Работа с базой данных

В моем проекте интерфейс хранилища реализует `postgres_food_repository.go`: создает, извлекает и удаляет записи. Благодаря тому что хранилище реализовано через интерфейс, в тестах можно подменить реальный PostgreSQL на in-memory реализацию, не меняя ни строчки в остальном коде. Все запросы используют параметризованные выражения, значения передаются отдельно от текста запроса

3.6 Пользовательский интерфейс

Интерфейс бота построен на inline-кнопках - это кнопки, которые прикрепляются непосредственно к конкретному сообщению, в отличие от обычных кнопок меню, которые закрепляются внизу чата. При нажатии inline-кнопка отправляет боту callback-запрос со строкой данных, которая определяет какое действие нужно выполнить.

Важная особенность реализации, бот не отправляет новое сообщение при каждом действии, а редактирует уже существующее. Это достигается через метод `EditMessageText` и `EditMessageReplyMarkup` в `go-telegram-bot-api`. Такой подход избавляет чат от мусора из старых сообщений и создает ощущение интерактивного интерфейса.

На практике это работает так: при нажатии «Отчёт за сегодня» бот отправляет одно сообщение с отчетом и кнопкой «Редактировать». При нажатии «Редактировать» у то же самого сообщения текст заменяется на список записей с кнопками удаления. После нажатия кнопки сообщение снова обновляется, но уже без недавно удаленной позиции.

Пользователь всё время работает с одним сообщением.

Клавиатуры формируются динамически: кнопки удаления генерируются в `KeyboardService` из реальных записей, полученных из базы данных. В callback-данные каждой кнопки записывается `edit: {id}` - идентификатор конкретной записи, по которому затем выполняется удаление.

3.7 Развёртывание через Docker

Запуск бота требует двух взаимосвязанных компонентов: самого приложения и базы данных PostgreSQL. Чтобы не зависеть от окружения конкретной машины и не настраивать всё вручную при каждом деплое, оба компонента упакованы в Docker-контейнеры.

Dockerfile — сборка образа бота

Dockerfile написан по принципу поэтапной сборки (multi-stage build).

Первый этап. За основу берётся официальный образ `golang:1.25-alpine`, который содержит минимум компонентов для компиляции кода. В этот же образ копируются файлы со списком зависимостей `go.mod` и `go.sum`, затем запускается команда установки этих зависимостей. В случае если в проект не добавлялись новые модули и эти файлы не изменялись, Docker не будет скачивать все библиотеки заново.

```
#Используем официальный образ для сборки  
FROM golang:1.25-alpine AS builder  
  
#Устанавливаем корневую директорию  
WORKDIR /app  
  
#Устанавливаем зависимости  
COPY go.mod go.sum ./  
RUN go mod download
```

Когда все зависимости установлены, копируем в образ остальной код и запускаем компиляцию и формирование бинарного файла - исполняемого файла, не требующего компиляции, готового к запуску, командой:

```
#Копируем остальной код  
COPY . .  
  
#Формируем бинарник  
RUN CGO_ENABLED=0 GOOS=linux go build -o /app/main ./main.go
```

Второй этап. Скачивается другой образ Alpine Linux, устанавливается рабочая директория в нее помещается бинарный файл, сформированный ранее. Инструкция ENTRYPOINT задаёт точку входа контейнера, указывая Docker, какой исполняемый файл необходимо запустить при старте контейнера.

```
#Установка образа
FROM alpine:latest

#Устанавливаем корневую директорию
WORKDIR /app

#Копируем бинарный файл
COPY --from=builder /app/main /app/main

ENTRYPOINT [ "./main" ]
```

docker-compose.yaml

Для оркестрации многокомпонентного приложения используется Docker Compose, который описывает два сервиса: базу данных PostgreSQL и непосредственно Telegram-бота, собранного ранее. Сервисы помещены в общую bridge-сеть, что позволяет боту обращаться к базе данных по имени контейнера tg-bot-db. Для сохранения данных БД при перезапусках используется том pgdata. Файл инициализации init.sql монтируется в специальную директорию образа PostgreSQL, что гарантирует автоматическое создание схемы базы данных при первом запуске.

```
services:

  tg-bot-db:
    image: postgres:16
    container_name: telegram-bot-db
    restart: always
    environment:
```

```

    - POSTGRES_PASSWORD=080813Pp
    - POSTGRES_USER=mpomy
    - POSTGRES_DB=tg_bot_db
  expose:
    - "5432"
  volumes:
    - pgdata:/var/lib/postgresql/data
    - ./init.sql:/docker-entrypoint-initdb.d/init.sql
  networks:
    - telegram-bot-net

bot:
  build: .
  container_name: telegram-bot
  restart: always
  depends_on:
    - tg-bot-db
  volumes:
    - ./config.yml:/app/config.yml
  networks:
    - telegram-bot-net

volumes:
  pgdata:

networks:
  telegram-bot-net:
    driver: bridge

```

Стоит отметить, что данные для подключения к базе данных “Вшиты” в файл, а не с помощью переменных окружения, что не является лучшей практикой, однако для контейнера не был задан внешний порт, что исключает внешнего подключения к базе данных.

Глава 4. Тестирование

4.1 Подход к тестированию

Тестирование приложения проводилось на двумя методами: автоматизированное модульное тестирование сервисного слоя и ручное функциональное тестирование пользовательских сценариев.

Такой подход обусловлен архитектурой проекта. Автоматизированные тесты целесообразны там, где есть четкий входной параметр и ожидаемый результат, в данном случае это алгоритм поиска продуктов. Поведение бота в целом: отправка сообщений, нажатие кнопок, редактирование записей, удобнее проверять вручную, имитируя реального пользователя.

4.2 Модульное тестирование

В Go тесты пишутся в файлах с суффиксом `_test.go` и запускаются встроенным инструментом `go test`. Никаких внешних библиотек не требуется, стандартная библиотека `testing` является достаточной.

В проекте используется подход табличных тестов (`table-driven tests`) - стандартная идиома Go-сообщества. Вместо того чтобы писать отдельную функцию для каждого сценария, все входные данные и ожидаемые результаты собираются в один срез структур, а затем перебираются в цикле. Это делает тест компактным и легко расширяемым — добавить новый сценарий означает добавить одну строку в таблицу.

Тесты покрывают функцию `findBestMatch` - сердце бизнес-логики. Для изоляции от реальной базы данных создается `mockRepo` и небольшой тестовый словарь из трёх позиций: «Яблоко», «Молоко 2.5%», «Лосось». Этого достаточно чтобы проверить все три уровня поиска независимо друг от друга.

Пять тестовых сценариев:

№	Сценарий	Ввод	Ожидаемый результат
1	Точное совпадение	яблоко	Яблоко
2	Поиск по подстроке	молоко	Молоко 2.5%
3	Продукт не найден	апельсин	ошибка
4	Опечатка — простой случай	Ласось	Лосось
5	Опечатка — не находит	молако	ошибка

Сценарий №5 требует отдельного пояснения. Расстояние Левенштейна между «молако» и «молоко» равно 1, что соответствует установленному в разделе 3.3 порогу `maxDistance = 1`, - следовательно, поиск должен был дать результат. Однако в базе данных продукт хранится под полным названием «Молоко 2.5%», и расстояние между «молако» и «молоко 2.5%» значительно превышает 1. Это граничный случай: когда пользователь вводит сокращённое название с опечаткой, алгоритм не находит совпадения, даже если оно кажется очевидным.

Тесты запускаются командой:

```
go test ./internal/service/...
```

4.3 Ограничения тестового покрытия

Автоматизированными тестами покрыт только алгоритм поиска. За рамками остались:

- Парсер ввода (ParseFoodInput) - функция извлекает название продукта и вес из текстового сообщения, обходя слова справа налево в поисках числа. Логика несложная, но граничные случаи — ввод без числа, несколько чисел в строке, пустая строка, тестами не закрыты, были протестированы в ручную.
- Расчёт порции (CalculatePortion) - чистая математическая функция без побочных эффектов, идеально подходит для юнит-тестов, но покрытия не требует из-за примитивности расчетов.
- Репозиторий - для тестирования SQL-запросов потребовалась бы тестовая база данных или mock-объект, реализующий интерфейс FoodRepository. Архитектура проекта это допускает, именно для этого репозиторий оформлен через интерфейс.

4.4 Функциональное тестирование

Функциональное тестирование проводилось вручную путём непосредственного взаимодействия с запущенным ботом. Проверялись все ключевые пользовательские сценарии.

Сценарий 1. Добавление продукта с точным совпадением

Вводится сообщение «Гречка 200». Бот корректно извлекает название и вес, находит продукт, рассчитывает КБЖУ и возвращает подтверждение с числовыми значениями. Запись появляется в дневном отчете.

Сценарий 2. Добавление продукта с несколькими совпадениями

Вводится «Творог 100». Поиск возвращает несколько вариантов, бот отображает inline-кнопки с названиями. После нажатия на конкретный вариант запись сохраняется, кнопки исчезают и приходит подтверждение.

Сценарий 3. Ввод с опечаткой

Вводится «Ласось 150». Алгоритм находит «Лосось» через расстояние Левенштейна и сохраняет запись корректно.

Сценарий 4. Просмотр дневного отчета

Через команду /menu открывается меню с кнопками. После нажатия «Отчёт за сегодня» бот формирует список всех записей за текущий день с суммарными показателями КБЖУ.

Сценарий 5. Редактирование и удаление записи

В режиме редактирования каждая запись отображается как кнопка с названием продукта. После нажатия запись удаляется из базы, список обновляется в том же сообщении без отправки нового. Кнопка «Закончить редактирование» возвращает к финальному отчету, возвращая на тоже место кнопку редактирования.

Сценарий 6. Продукт не найден

Вводится название продукта, которого нет в базе. Бот сообщает что продукт не найден и не создает запись.

Все сценарии отработали в соответствии с ожидаемым поведением.

4.5 Логирование

В приложении реализовано логирование с помощью стандартного пакета log. Логи покрывают весь жизненный цикл приложения: старт и подключения к базе данных, каждая операция сохранения, удаления и поиска продуктов, будь то успешная или нет.

В тексте сообщений используются явные префиксы уровней: DEBUG, INFO, ERROR. Это позволяет при просмотре вывода быстро отделить отладочную информацию от значимых событий и ошибок. Например, при каждом запросе к базе данных в лог выводится точный диапазон дат выборки - это помогает диагностировать проблемы с

часовыми поясами. При поиске продукта фиксируется количество найденных вариантов. При критических сбоях полный текст ошибки с контекстом.

Текущая реализация имеет существенное ограничение: все сообщения выводятся только в стандартный поток вывода контейнера (stdout). При перезапуске контейнера вся история логов теряется. Это приемлемо на этапе разработки, но недостаточно для длительной эксплуатации. Диагностика ошибок, произошедших в прошлом, становится невозможной.

Кроме того, стандартный пакет log не поддерживает структурированное логирование: каждое сообщение это просто строка. Современные подходы предполагают вывод логов в формате JSON с фиксированными полями (время, уровень, сообщение, контекст), что позволяет автоматически их индексировать и фильтровать.

Заключение

В ходе выполнения выпускной квалификационной работы разработан Telegram-бот для автоматизированного учёта питания и расчета КБЖУ. Все задачи, поставленные во введении, выполнены:

- проведен анализ предметной области и обзор существующих решений;
- спроектирована слоистая архитектура приложения;
- разработан алгоритм нечеткого поиска продуктов на основе расстояния Левенштейна;
- реализовано хранение данных пользователей в PostgreSQL;
- разработан пользовательский интерфейс на основе inline-кнопок;
- обеспечено развертывание системы через Docker;
- проведено модульное и функциональное тестирование.

Бот работает, решает поставленную задачу: пользователь вводит название продукта и вес порции текстом, система рассчитывает КБЖУ и сохраняет запись. Алгоритм нечеткого поиска снижает требования к точности ввода, незначительные опечатки не приводят к ошибке. Отсутствие обязательной регистрации и работа внутри Telegram снижают порог входа для новых пользователей.

Среди текущих ограничений следует выделить: база продуктов насчитывает около 75 позиций, логи не сохраняются между перезапусками контейнера, тестовое покрытие ограничено сервисным слоем.

В качестве направлений дальнейшего развития рассматриваются: расширение базы продуктов, добавление персональных норм КБЖУ, подключение системы хранения логов, разработка Telegram Mini App для расширенной аналитики рациона, а также деплой на выделенный сервер.

Список литературы

1. Пищевые привычки россиян меняются: растёт спрос на полезный рацион и обогащенные продукты. — Текст : электронный. — URL: <https://marketing.rbc.ru/articles/16337/> (дата обращения: 20.05.2026).
2. Здоровое питание : информационный бюллетень ВОЗ. — Текст : электронный. — URL: <https://www.who.int/ru/news-room/fact-sheets/detail/healthy-diet> (дата обращения: 20.05.2026).
3. Подсчёт КБЖУ : результаты общероссийского опроса / Russian Field. — Текст : электронный. — URL: <https://russianfield.com/calories> (дата обращения: 20.05.2026).
4. Дневник питания: что это, как правильно вести, шаблон, приложения с подсчетом калорий. — Текст : электронный. — URL: <https://sportmail.ru/article/others/67587753/> (дата обращения: 20.05.2026).
5. Research Identifies Barriers in Tracking Meals / Today's Dietitian. — Текст : электронный. — URL: <https://www.todaysdietitian.com/research-identifies-barriers-in-tracking-meals/> (дата обращения: 20.05.2026).
6. Язык программирования Golang: плюсы и минусы, применение / Яндекс Практикум. — Текст : электронный. — URL: <https://practicum.yandex.ru/blog/zachem-uchit-yazyk-go/> (дата обращения: 20.05.2026).
7. PostgreSQL — что такое PostgreSQL: основные принципы работы и возможности базы данных / Skyeng. — Текст : электронный. — URL: <https://skyeng.ru/magazine/cto-takoe-postgresql/> (дата обращения: 20.05.2026).

8. PostgreSQL: что это, для чего нужна, основы и преимущества / Яндекс Практикум.
— Текст : электронный. — URL:
<https://practicum.yandex.ru/blog/что-такое-subd-postgresql/> (дата обращения: 20.05.2026).
9. Что такое утиная типизация / Журнал «Код». — Текст : электронный. — URL:
<https://thecode.media/duck-type/> (дата обращения: 20.05.2026).
10. Расстояние Левенштейна для чайников / Хабр. — Текст : электронный. — URL:
<https://habr.com/ru/articles/676858/> (дата обращения: 20.05.2026).
11. Multi-stage builds / Docker Documentation. — Текст : электронный. — URL:
<https://docs.docker.com/build/building/multi-stage/> (дата обращения: 20.05.2026).
12. Основы тестирования Go: простые тесты, testify и визуализация покрытия / Хабр.
— Текст : электронный. — URL: <https://habr.com/ru/companies/otus/articles/739468/>
(дата обращения: 20.05.2026).
13. Add a test / The Go Programming Language. — Текст : электронный. — URL:
<https://go.dev/doc/tutorial/add-a-test> (дата обращения: 20.05.2026).